Jade Muyambo

M.S Data Science

University of Miami

2026 ASA South Florida Student Data Challenge

1. Executive Summary

This report describes the full workflow used to predict LBDHDD_outcome (noise-perturbed HDL cholesterol, mg/dL) for the ASA South Florida Student Data Challenge. I completed this project using Python in Jupyter Notebook. The main Python libraries I used were pandas, NumPy, and scikit-learn. The modeling pipeline includes data cleaning, numeric imputation, baseline benchmarking, gradient-boosting hyperparameter tuning, and a stacked ensemble for the final submission. The final submission file, pred.csv, contains one column named pred with 200 predictions in the same order as the provided test set. The stacking model was selected to generate the final predictions after repeated cross-validation as it achieved a lower RMSE than both the Ridge baseline and the tuned HistGradientBoosting model.

2. Software and Programming Environment

I completed this project in Python using a Jupyter Notebook environment.

The main Python libraries used were:

- Pandas and NumPy for data loading and manipulation
- Scikit-learn for preprocessing, model training, validation, and tuning

Key scikit-learn tools used were:

- Pipeline
- ColumnTransformer
- SimpleImputer
- RepeatedKFold
- cross_val_score
- RandomizedSearchCV
- Ridge, ElasticNet
- HistGradientBoostingRegressor
- ExtraTreesRegressor
- StackingRegressor

This workflow allowed me to keep preprocessing and modeling together in pipelines, which helps avoid leakage and ensures consistent cross-validation.

3. Data Overview and Preprocessing

The competition dataset is based on NHANES 2024 and contains a training set with 1,000 observations, test set with 200 observations, and variables with 97 columns in training, including the target. The prediction target is LBDHDD_outcome representing noise-perturbed HDL cholesterol (mg/dL). The provided files used were train.csv, test.csv, and variable_labels.csv.

Initially, when I loaded all three files into pandas DataFrames and checked their dimensions, train.csv shape was (1000, 97), test.csv shape was (200, 96), and variable_labels.csv shape was (96, 2). Both the training and test datasets contained a column named Unnamed: 0, which functioned as an index column rather than a true predictor. I removed this column from both datasets to avoid introducing artificial structure into the model. After dropping the Unnamed: 0 column, the train shape became (1000, 96) and the test shape became (200, 95).

I separated the target variable from the predictors in order to train the model correctly, avoid leakage, and get a trustworthy RMSE. To prevent data leakage, the target variable (LBDHDD_outcome) was removed from the feature matrix before any preprocessing, cross-validation, or model fitting. In addition, preprocessing was placed inside scikit-learn pipelines so that imputation parameters were learned only from each training fold during cross-validation, not from the validation fold. X contains all columns except for LBDHDD_outcome and y contains LBDHDD_outcome. This produced X shape (1000, 95) and y. shape (1000,). I aligned the test set columns to the training feature columns using test_aligned = test.reindex(columns=X.columns) to ensure the prediction file used the exact same feature order as the training data. This step is important because the competition requires the predictions to be in the same order as the test dataset and generated from the same predictor structure.

All 95 predictors were numeric in this dataset, so I used a simple and robust numeric preprocessing pipeline: median imputation (SimpleImputer(strategy="median")). I chose median imputation because it is stable for skewed biomedical and nutrition variables and avoids distortion from extreme values.

4. **Validation and Tuning Strategy**

I used RMSE as the evaluation metric throughout the model deployment since the competition ranking is based on RMSE. I created a custom RMSE scoring function using make_scorer and mean_squared_error, then used it consistently across all cross-validation and tuning steps. For the cross-validation design, I used Repeated K-Fold Cross-Validation, n_splits=5, and n_repeats=5. This results in 25 validation folds total, which gives a more stable estimate of model performance than a single train and test split. This was especially useful because the training dataset has only 1,000 observations. This validation design helped reduce the risk of overfitting to one particular split and made model comparisons between baseline, tuned, and stacked more reliable.

Validation setup code snipet:

```
def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))
rmse_scorer = make_scorer(
    lambda yt, yp: np.sqrt(mean_squared_error(yt, yp)),
```

```
    greater_is_better=False
)
cv = RepeatedKFold(n_splits=5, n_repeats=5, random_state=RANDOM_STATE)
```

5. Model Development and Performance

Model	CV RMSE (mean)	CV RMSE SD	Notes
Ridge baseline	6.058	0.344	Linear benchmark with median imputation
Tuned HistGradientBoosting	4.839	0.255	Best single tuned non-linear model
Stacking ensemble	4.708	0.226	Combined HGB, Ridge, ElasticNet, ExtraTrees

CV RMSE (mean) and CV RMSE SD numbers are rounded

I used a staged modeling approach to build performance gradually and keep the process interpretable. I started with a Ridge Regression model as a baseline. This provides a strong linear benchmark and is easy to interpret. The baseline pipeline includes median imputation and ridge regression. The cross-validated baseline performance showed the baseline ridge CV RMSE mean of 6.05805188599999 and standard deviation of 0.343833918982501. This baseline established a reference point for evaluating more advanced models.

For the main nonlinear model, HistGradientBoostingRegressor (HGB), the dataset includes many numeric predictors such as dietary, demographic, and anthropometric variables. I expected nonlinear relationships and interactions. I trained a HistGradientBoostingRegressor, which is efficient and often strong on tabular data, to capture those. I used RandomizedSearchCV with the same RMSE scorer and repeated K-Fold CV setup. I sampled 80 parameter combinations, which provided a good balance between exploration and runtime. The search space included learning_rate, max_depth, max_leaf_nodes, min_samples_leaf, l2_regularization, max_bins, early_stopping, and validation_fraction. The best HGB CV RMSE results for the mean is 4.8388049739515875 and standard deviation is 0.2545. This was a large improvement over the Ridge baseline, indicating that nonlinear modeling substantially improved prediction quality.

To further improve performance, I built a stacking ensemble that combines multiple models with different strengths. For the base learners I used tuned HistGradientBoosting pipeline, which is the best model from RandomizedSearchCV, Ridge Regression pipeline, ElasticNet pipeline, and ExtraTreesRegressor pipeline. For meta-learner I used ridge regression as the final estimator. I used stacking because it can improve prediction accuracy by combining nonlinear boosted trees (HGB), linear shrinkage models (Ridge and ElasticNet), and a high-variance tree ensemble (ExtraTrees). These models capture different signal patterns, and the meta-learner learns how to weight them. The stacking performance CV RMSE for mean is 4.707736641206233 and

standard deviation of 0.22589936568970065. This improved performance over the tuned HGB model by about 0.13 RMSE.

Stacking ensemble model code snipet:

Due to the 4-page report limit, the full stacking implementation is provided in the submitted notebook. A screenshot of the final stacking setup is shown below for reference.

[84]:
```
hgb_best_pipe = clone(best_model)

ridge_pipe = Pipeline(steps=[
    ("prep", preprocess),
    ("model", Ridge(alpha=1.0))
])

enet_pipe = Pipeline(steps=[
    ("prep", preprocess),
    ("model", ElasticNet(alpha=0.01, l1_ratio=0.2, max_iter=10000, random_state=RANDOM_STATE))
])

et_pipe = Pipeline(steps=[
    ("prep", preprocess),
    ("model", ExtraTreesRegressor(
        n_estimators=500,
        max_depth=None,
        min_samples_leaf=2,
        random_state=RANDOM_STATE,
        n_jobs=-1
    ))
])

stack_model = StackingRegressor(
    estimators=[
        ("hgb", hgb_best_pipe),
        ("ridge", ridge_pipe),
        ("enet", enet_pipe),
        ("et", et_pipe)
    ],
    final_estimator=Ridge(alpha=1.0),
    cv=5,
    n_jobs=-1,
    passthrough=False
)

stack_scores = cross_val_score(stack_model, X, y, scoring=rmse_scorer, cv=cv, n_jobs=1)

print("Tuned HGB CV RMSE:", -search_hgb.best_score_)
print("Stacking CV RMSE:", -stack_scores.mean(), "+/-", stack_scores.std())
```

```
Tuned HGB CV RMSE: 4.8388049739515875
Stacking CV RMSE: 4.707736641206233 +/- 0.22589936568970065
```

6. Final Model Selection

I selected the model with the lower cross-validated RMSE. Lower RMSE is better because it means the model's predictions are closer to the actual HDL values. The tuned HistGradientBoosting model improved substantially over the Ridge, and the stacking ensemble further reduced RMSE by about 0.13 compared with the HGB model. Therefore, the stacking model with the lower RMSE was the best final model to use for generating predictions on the test set.

7. Code Repository

I have submitted my notebook source file alongside this report.

8. Conclusion

Future improvements could include broader hyperparameter search, feature interaction engineering, and repeated tuning of the stacking meta-learner, but the current approach balanced predictive performance and runtime effectively.